ITGE 272
Professional Productivity Application

Lab11 (Individual): Collaborative coding
Due: By the end of lab (12.30PM)

Task 00: Workspace Setup and Initial Configuration

1. **Create an Assignment Workspace:**
   a. Create a new folder in your machine (e.g., /Documents/lec11-git-lab/) to serve as your project workspace.
2. **Configure Git User Credentials (terminal):**
   a. Create a GitHub account on https://github.com/. You can skip this step if you already have GitHub account
   b. Open your terminal and configure your name and email. These credentials will be attached to every commit you make. Change Your name to your GitHub's user name and email address to your GitHub's account address.

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

   c. Ensure Git for Windows is configured to use the Windows Credential Manager (GCM) for secure credential storage. This allows you to avoid re-entering your Personal Access Token or password repeatedly.
   In terminal type:

```
git config --global credential.helper manager
```

   Notes: (This is often the default if using Git for Windows; the first push over **HTTPS** will prompt the GCM.)

Note: If you plan to complete every task using only the GitHub Web UI (Part A), you can skip the git config and Credential Manager setup above — those steps are only needed for the Command Line path (Part B).

Task 01: Basic Repository Initialization and Remote Push

Objective: Initialize a local Git repository, track changes, commit them, and push the entire project to a new remote repository on GitHub.

Choose your path: complete Task 1 using Part A (GitHub Web UI) if git is blocked on your machine, or Part B (Command Line) otherwise. Either path satisfies the deliverable below.

Part A: GitHub Web UI

Step	Action	Explanation	GitHub UI Navigation
W1.1	Create a new repository.	Set up an empty remote repository on GitHub to hold your project.	*"+" icon (top right) → "New repository"*
W1.2	Name and create the repository.	Name it lab11 and check "Add a README file" so the repo isn't empty — this replaces git init + the first commit.	*Repository name: lab11 → check "Add a README file" → "Create repository"*

Step	Action	Explanation	GitHub UI Navigation
W1.3	Confirm the repository is live.	GitHub automatically creates README.md and an initial commit for you — this replaces git remote add + git push.	*View the repository's file list and commit history*

Part B: Command Line

Step	Action	Explanation	Example Command
1.1	Initialize a local Git repository.	Start tracking changes in your lab11 folder.	*git init*
1.2	Create a simple code file.	Create a file named README.md and add some content (e.g., # Lab 11: Git Basics).	*echo "# Lab 11: Git Basics" > README.md*
1.3	**Stage** the new file.	The git add command moves changes from the working directory to the staging area.	*git add README.md*
1.4	**Commit** the staged changes.	The git commit command creates a snapshot of the proiejct	*git commit -m "Initial commit of project structure"*
1.5	Create a new **remote repository** on GitHub.	Create an **empty** repository on GitHub.com (do **not** check the "Initialize this repository with a README" box).	*Action: Create new repo on GitHub UI*
1.6	Link the local repository to the remote.	Add the GitHub repository as a remote named origin.	*git remote add origin https://github.com/YourUser/lab11.git*
1.7	**Push** changes to the remote.	The git push command uploads your local commits to the remote repository. The -u flag sets the upstream tracking branch.	*git push -u origin main*

Deliverable for Task 1

1. **A single hyperlink** to your remote GitHub repository (https://github.com/YourUser/lab11).
2. The repository must contain the **README.md** file and the history must show the "Initial project setup..." commit

Task 02: Basic Branching, Remote Tracking, and Merging
Objective: Demonstrate isolated development using local and remote branches and integrate changes back into the main codebase via a standard merge.
Choose your path: complete Task 2 using Part A (GitHub Web UI) or Part B (Command Line). Either path satisfies the deliverable below.

Part A: GitHub Web UI

Step	Action	Explanation	GitHub UI Navigation
W2.1	Create a new branch.	Use a branch for isolated development of a new feature.	*"Branch" dropdown (shows "main") → type feature/contact-page → "Create branch"*
W2.2	Add contact.html on the branch.	Create the "feature" file directly on your branch.	*Confirm you're on feature/contact-page → "Add file" → "Create new file" → name contact.html, content Contact Info → "Commit changes"*
W2.3	Open a Pull Request.	Propose merging your branch into main for review.	*"Pull requests" tab → "New pull request" → base: main, compare: feature/contact-page → "Create pull request"*
W2.4	Review and merge the Pull Request.	Integrate the completed feature into the stable main branch.	*"Files changed" tab to review → "Merge pull request" → "Confirm merge"*
W2.5	(Optional) Delete the branch.	Keep the branch list tidy now that it's merged.	*"Delete branch" button on the merged PR page*

Part B: Command Line

Step	Action	Explanation	Example Command
2.1	Create a new local **feature branch**.	Use a branch for isolated development of a new feature.	*git checkout -b feature/contact-page*
2.2	Make and commit changes on the feature branch.	Create a new file, **contact.html**, and commit the "feature" locally.	*echo " Contact Info " > contact.html* *git add .* *git commit -m "FEAT: Add contact page"*

Step	Action	Explanation	Example Command
2.3	**Push** the local branch to the remote.	Create a corresponding branch on GitHub and set it up to track the remote.	*git push -u origin feature/contact-page*
2.4	Switch back to the **main** branch.	Return to the stable code base.	*git checkout main*
2.5	**Merge** the feature branch locally.	Integrate the completed feature into the stable main branch.	*git merge feature/contact-page*
2.6	Push the final merged result to the remote.	Update the remote main branch on GitHub with the new history.	*git push origin main*

Deliverables for task 2

1. A single hyperlink to the main branch of your GitHub repository.
2. The repository's main branch must contain the contact.html file, and the commit history must clearly show the merge commit from feature/contact-page.

Task 03 (Optinal for an extra credit): Multiple Branches and Conflict Resolution via Pull Request

Objective: Demonstrate proficiency in a complex, real-world scenario involving concurrent development, a critical hotfix, and resolving a merge conflict within a Pull Request workflow. (Max 15 extra points)

Scenario Overview

You are working on a feature (feature/refactor-config), but a critical bug is found on the stable code (main). You must create a hotfix, merge it, and then integrate those changes back into your in-progress feature branch, which will cause a conflict you must resolve before submitting your final work for review via a **Pull Request**.

This task is optional. You may complete it using Part A (GitHub Web UI) or Part B (Command Line) — both lead to the same deliverable below, so pick whichever you're more comfortable with.

Part A: GitHub Web UI

Action	Explanation	GitHub UI Navigation (or Action)
Create config.txt on main.	Create a simple configuration file with three lines directly on the default branch.	*"Add file" → "Create new file" → name config.txt, paste VERSION=1.0 / PORT=8080 / API_KEY=abc → commit directly to main*
Start the long-running feature.	Branch off of main to create the feature branch.	*Branch dropdown → type feature/refactor-config → "Create branch: … from 'main'"*
Feature work (introduce	Edit Line 1 of config.txt to VERSION=2.0 and commit this change	*On feature/refactor-config: edit config.txt → "Commit changes" (commit directly to this branch)*

Action	Explanation	GitHub UI Navigation (or Action)
future conflict).	directly to the feature branch. Do NOT open a Pull Request yet.	
HOTFIX: create a hotfix branch.	A critical bug requires a hotfix on main. Branch from main again, separately from your feature branch.	*Branch dropdown (on main) → type hotfix/fix-api-key → "Create branch"*
Hotfix work, merge into main.	Edit Line 3 of config.txt to fix the API key: API_KEY=xyz. Commit on the hotfix branch, then merge it into main via a Pull Request.	*Edit + commit on hotfix/fix-api-key → "New pull request" (hotfix/fix-api-key → main) → "Merge pull request"*
Re-engage feature work (create a merge conflict).	Open a Pull Request from your feature branch into main. Because main advanced (from the hotfix) since you branched, GitHub will flag a conflict on config.txt.	*"New pull request" → base: main, compare: feature/refactor-config → "Create pull request"*
Resolve the Merge Conflict.	Manually edit config.txt to keep both your VERSION=2.0 change (Line 1) and the API_KEY=xyz hotfix (Line 3).	*On the PR page: "Resolve conflicts" → edit the file, removing conflict markers → "Mark as resolved" → "Commit merge"*
Create and Merge Pull Request.	Finish merging your resolved feature branch into main.	*"Merge pull request" → "Confirm merge"*

Part B: Command Line

Action	Explanation	Example Command (or Action)
Create config.txt on main.	Create a simple configuration file with three lines. Commit and push it to remote main.	echo -e "VERSION=1.0\nPORT=8080\nAPI_KEY=abc" > config.txt ...then push...
Start the long-running feature.	Branch off of main to create the feature branch.	git checkout -b feature/refactor-config

Action	Explanation	Example Command (or Action)
Feature work (Introduce future conflict).	Edit **Line 1** of config.txt to **VERSION=2.0**. **Commit** this change, but **DO NOT push yet.**	*Edit config.txt* git commit -m "Refactor: Start work on V2 configuration"
HOTFIX: Switch to main and create a hotfix branch.	A critical bug requires a hotfix on main. Switch back to the stable branch first.	git checkout main git checkout -b hotfix/fix-api-key
Hotfix work.	Edit **Line 3** of config.txt to fix the API key: **API_KEY=xyz**. **Commit** and **push** the hotfix branch to remote.	*Edit config.txt* git commit -m "Hotfix: Corrected API key value" git push -u origin hotfix/fix-api-key
Merge the hotfix into main.	Merge the hotfix into the stable branch locally and push the updated main to remote.	git merge hotfix/fix-api-key git push origin main
Re-engage feature work (Create a merge conflict).	Switch back to your feature/refactor-config branch. **Merge the updated main** into your feature branch to pull in the hotfix. This will cause a conflict on config.txt (due to your work in 3.3).	git checkout feature/refactor-config **git merge main**
Resolve the Merge Conflict.	Manually edit config.txt to keep both your **VERSION=2.0** change (Line 1) and the **API_KEY=xyz** hotfix (Line 3). Stage and commit the resolution.	*Edit config.txt*, resolving all conflict markers. git add config.txt git commit -m "Resolved conflict from main hotfix"
Push the resolved feature branch to remote.	Update your feature branch on GitHub with the conflict resolution.	git push origin feature/refactor-config

Action	Explanation	Example Command (or Action)
Create and Merge Pull Request.	On GitHub, create a Pull Request from feature/refactor-config to main. Merge the Pull Request.	*Action: Create and merge PR on GitHub UI*

Deliverable for Task 3

1. **A single hyperlink** to the **merged Pull Request** on GitHub (i.e., the PR page that shows it was successfully merged).
2. The final **config.txt** file in the main branch of your repository must contain the following content, demonstrating that all changes were successfully merged:

```
VERSION=2.0

PORT=8080

API_KEY=xyz
```

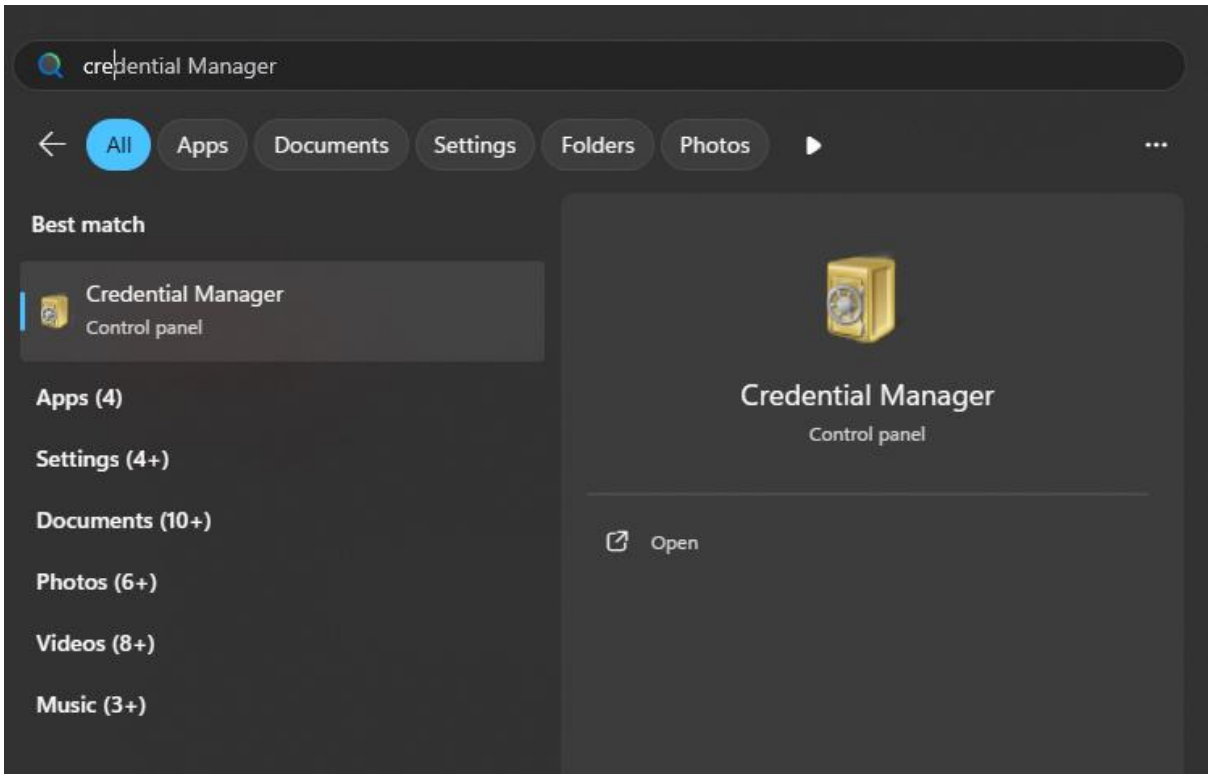
Notes: After finished the assignment. You may want to delete your login credentials in a public machine (which is strongly recommended to do)

1. To do that open terminal and type

```
git config --global --unset user.name

git config --global --unset user.email
```

2. After that, type credential manger on widows start and open it



3. Click on windows credentials and search for github credentendials and click remove

